RESEARCHLY AI · R26-IT-116

Module 3
Research Data Collection & Management

Scraping & ETL pipeline, topic categorization, point-wise summarization, and plagiarism-trend analysis — architecture, ML methodology, and calculation logic

Module owner: N. V. Hewamanne

Service: `services/module3-data` (FastAPI, port 8003)

Frontend: `apps/web/src/app/(dashboard)/data-management/*`

Document generated: 27 August 2026

Contents

1 · Overview

Module 3 is the **data backbone** of the platform — every other module's ML features ultimately depend on the corpus this module builds and maintains. It covers:

1. **Data pipeline / ETL** — background scraper jobs that pull papers from external sources (arXiv, Semantic Scholar, and others) into Supabase.
2. **Topic categorization** — multi-label classification of a paper's research category.
3. **Point-wise summarization** — turns a full paper (text or uploaded PDF) into a structured, section-aware bullet summary.
4. **Plagiarism-trend analysis** — topic-level plagiarism trend buckets over time, plus direct pairwise comparison of two papers.
5. **Data quality metrics** — completeness/consistency/duplicate-rate reporting over the whole scraped corpus.

All five live in one FastAPI microservice (`services/module3-data` , port `8003`), consumed by four pages under `apps/web/src/app/(dashboard)/data-management/` (`pipeline` , `categorization` , `summarizer` , `plagiarism-trends`).

> **Central design decision:** Module 3 owns and encodes the 4,219-paper SLIIT corpus (`data/papers_raw_sliit.json` + `data/paper_embeddings.npy`) once, in-memory, and shares that SBERT index across topic categorization, plagiarism trends, and (indirectly, via a shared model file) Module 1's similar-papers/gap-analysis features. This avoids re-encoding the same corpus multiple times across modules and keeps every module's "real paper" evidence consistent.

2 · System Architecture & Data Flow

```
Next.js page (data-management/*) │ fetch() with Supabase JWT ▼ Express API Gateway (apps/api-gateway, port 3001) │ requireAuth → mlRateLimiter → callMlService(3, path, method, body) [150s timeout — first-load encode] ▼ FastAPI Module 3 service (services/module3-data, port 8003) │ router → service layer (topic_classifier / extractive_summarizer / │ plagiarism_analyzer / paper_index) → scrapers (background jobs) ▼ ┌── models/trained_topic_classifier/classifier.pkl (TF-IDF + LogReg bundle) ├── models/trained_plagiarism_analyzer/trend_index.pkl (topic buckets) ├── models/sbert_plagiarism/ (SLIIT fine-tuned SBERT, shared with Module 1) ├── data/papers_raw_sliit.json + paper_embeddings.npy (4,219-paper index) ├── Supabase Postgres (research_papers, research_summaries, plagiarism_trends) └── Gemini 2.5 Flash (categorization + summarization fallback only)
```

The gateway route file is `apps/api-gateway/src/routes/module3.routes.ts` (282 lines — the largest of the four, owing to PDF-upload multipart passthrough for both the summarizer and the plagiarism comparator). Module 3 is given a longer proxy timeout (150s vs. 60s for the other three modules) because the first request after a cold start triggers a one-time ~90-second encode of the full 4,219-paper corpus if the pre-computed `paper_embeddings.npy` is missing.

3 · Feature A — Data Pipeline (Scraping / ETL)

Async scraper jobs, no ML

What it does

The Pipeline page lets an admin/user kick off a background scrape job for a chosen source (`POST /data/scrape`) and poll its status (`GET /data/scrape/{job_id}`) — papers collected, current status, and any error.

Why this exists as a separate ETL layer

Every downstream ML feature — categorization, summarization, plagiarism, gap analysis, proposal generation, supervisor matching — depends on having a large, real, continuously-refreshable corpus of research papers. Centralising ingestion here (rather than each module scraping its own data) keeps `research_papers` as a single source of truth with consistent schema and embeddings.

How it is implemented (`app/routers/pipeline.py`)

- Six source scrapers exist under `app/scrapers/` (IEEE, arXiv, ACM, SLIIT, Scholar, Semantic Scholar), each subclassing a common `base_scraper.Paper` dataclass shape; at present the router only wires up `arxiv` and `semantic_scholar` as callable (`_get_scraper`) — requesting another source returns a 400 with the list of currently available ones.
- Jobs run as FastAPI `BackgroundTasks` against an in-memory `_jobs` dict keyed by a short UUID — status transitions *queued → running → success/failed*. With no explicit `query` , arXiv scrapes across 11 pre-defined categories and Semantic Scholar across 13 pre-defined topics, splitting `max_papers` roughly evenly per bucket (`max_papers // 11` / `// 13`) so a broad crawl still gets topical diversity rather than exhausting the quota on one category.

4 · Feature B — Topic Categorization

TF-IDF + One-vs-Rest Logistic Regression | Gemini fallback

What it does

Given a paper's text, `POST /data/categorize` returns every category whose confidence clears a threshold, a confidence score per category, the top-K categories regardless of threshold (for a "did you mean" UI), and up to 6 related real SLIIT papers in the same space.

Why TF-IDF + linear classification (not a transformer)

Multi-label topic classification over a fixed, moderate-size label set is a well-understood problem where a classical TF-IDF vectoriser + One-vs-Rest logistic regression bundle already reaches strong accuracy, trains in seconds on CPU, and returns calibrated per-label probabilities directly from `predict_proba` — with none of a transformer's GPU/latency overhead for what is, in production use, a request-time classification of short text.

How it is implemented (`app/services/topic_classifier.py`)

1. Loads `models/trained_topic_classifier/classifier.pkl` — a bundle containing a fitted `vectorizer` (TF-IDF), a `classifier` (One-vs-Rest logistic regression), and the ordered `labels` list.
2. Input text is TF-IDF transformed (`vec.transform([text])`) and passed to `clf.predict_proba(X)[0]` to get a probability per label; if the underlying estimator has no `predict_proba` (non-probabilistic OvR), the raw decision-function score is passed through a manual sigmoid: `1 / (1 + e^(-score))` .
3. Labels are sorted descending by probability; the top-K (regardless of threshold) become `top_categories` , and every label at or above the caller's `threshold` (default 0.3) becomes the returned `categories` list.

If a `paper_id` is supplied, the resulting categories are written back to `research_papers.keywords` — so classifying a paper also tags it for future retrieval/filtering. If the local bundle is missing, the router falls back to a Gemini prompt requesting the same `{classifications: [{category, confidence}]}` shape.

5 · Feature C — Point-Wise Summarizer

Extractive SBERT (centroid + lead-bias + MMR) **Gemini abstractive fallback**

What it does

Given a paper's full text (pasted or PDF-uploaded), `POST /data/summarize` / `/summarize/upload` return a length-controlled bullet summary (*quick / standard / detailed / extensive*, 5–18 points), each bullet tagged with a section category (*background / objective / methodology / results / limitations / conclusion*), plus a compression ratio and a downloadable styled HTML report.

Why extractive, not abstractive (BART/T5)

The spec called for a `facebook/bart-large-cnn` or `flan-t5-base` + LoRA abstractive summarizer. In production the live path is **extractive** instead — every returned sentence is *verbatim from the source paper*. The code's own docstring states the rationale directly: no fine-tuned BART weights are bundled for the SLIIT domain, extractive output carries *zero hallucination risk* for academic use (a summary that invents a claim the paper never made would be far worse than one that's merely less fluent), and it runs in ~80ms per paper on CPU versus ~5s for a BART forward pass. A Gemini abstractive fallback exists for when the extractive pipeline itself fails.

How it is implemented (`app/services/extractive_summarizer.py`)

1. **Sentence splitting** — regex sentence-boundary split, keeping only 20–600 character sentences, capped at 80 sentences total (`MAX_SENTENCES`) to bound encoding time.
2. **Encoding** — every sentence is embedded with the SLIIT-fine-tuned SBERT (shared with Module 1's `models/sbert_plagiarism`); the corpus **centroid** (mean embedding, re-normalised) represents the "topical core" of the whole document.
3. **Per-sentence base score** — a weighted sum of three signals:

```
centroid_sim = embedding(sentence) · centroid (cosine, since both normalised) position_score = 1.0
if sentence in first 20% of document (lead-bias) 0.7 / 0.5 if in last 10% / 25% (papers often
restate key points near the end) linearly decays otherwise length_score = 1.0 for 60–250 chars;
linear penalty outside that range (too short = fragment, too long = rambling) base =
0.55·centroid_sim + 0.20·position_score + 0.10·length_score
```

4. **Greedy MMR selection** (Maximal Marginal Relevance) — sentences are picked one at a time; at each step, every remaining candidate is scored as `base[i] − 0.15·redundancy` , where `redundancy = max cosine similarity to any sentence already selected` . This is the "novelty" term (weight 0.15 mentioned in the module's own docstring) — it actively penalises picking near-duplicate sentences, so the summary doesn't repeat the same point twice even if it scored highly on relevance alone. Selection continues until the length preset's target point-count is reached.
5. **Section categorisation** — each selected sentence is matched against six ordered regex families (`_CATEGORY_PATTERNS` , checked most-specific first: conclusion → limitations → results → methodology → objective → background), falling back to a position heuristic (first 15% → background, last 15% → conclusion, else "general") when no keyword pattern matches.
6. Selected sentences are re-sorted back into document order for the flat `summary` string, and grouped by category (in a fixed display order) for the frontend's sectioned bullet view.

6 · Feature D — Plagiarism Trend Analysis

SBERT topic buckets + n-gram overlap

What it does

Two related capabilities: (1) `POST /plagiarism-trends/search` — given a topic, find the closest pre-computed "topic buckets" and their year-by-year similarity trend (rising/falling/stable), alongside actual related SLIIT papers; and (2) `POST /plagiarism-trends/compare` / `compare-pdf` — directly compare two arbitrary paper texts (or PDFs) for similarity, returning a risk level and the most similar sentence pairs between them.

Why two different techniques for one feature

Trend search answers *"how much has plagiarism/overlap been a problem in this topic area historically?"* — a corpus-level, pre-computable question well suited to a static SBERT index of topic buckets. Pairwise comparison answers *"are these two specific documents too similar?"* — a request-time question that needs both semantic similarity (to catch paraphrasing) *and* exact overlap (to catch verbatim copy-paste), so it combines two signals rather than relying on embeddings alone.

How trend search is implemented (`app/services/plagiarism_analyzer.py::search_trends`)

Loads `models/trained_plagiarism_analyzer/trend_index.pkl` — pre-computed topic-bucket embeddings plus metadata (`n_records` , per-year avg/max similarity, flagged high-similarity pairs). The query topic is embedded and cosine-scored against every bucket (`_INDEX["embeddings"] @ qvec`); buckets below `min_topic_similarity` (default 0.25) are dropped, and the surviving top-K are returned alongside real related SLIIT papers pulled from the shared paper index (Section 8) so the user always sees concrete evidence even when no bucket is a close match.

How pairwise comparison is implemented (`compare_papers`)

1. **Document-level similarity** — both full texts are embedded (batch of 2) and their cosine similarity taken directly.
2. **4-gram Jaccard overlap** — both texts are tokenised and turned into overlapping 4-word n-gram sets; `jaccard = |A ∩ B| / |A ∪ B|` , plus one-sided overlap ratios `|A∩B|/|A|` and `|A∩B|/|B|` to show which document contains more of the shared material.
3. **Flagged sentence pairs** — both documents are sentence-split (capped at 60 sentences each), all sentences from both are embedded in a single batched call, and a full similarity matrix $M = \text{sentences\_A} \cdot \text{sentences\_B}^{T}$ is computed. The top-K highest-similarity cells (via `np.argpartition` , avoiding a full sort) are surfaced as flagged pairs, stopping early once similarity drops below 0.5.

```
risk_score = 0.75 · document_similarity + 0.25 · ngram_jaccard (SBERT weighted higher —
paraphrasing is the more common real-world plagiarism form; n-grams remain a copy-paste
"tripwire") risk_level = "high" if risk_score ≥ 0.80 "medium" if risk_score ≥ 0.60 "low" if
risk_score ≥ 0.30 "minimal" otherwise
```

7 · Feature E — Data Quality Metrics

Deterministic aggregation, no ML

What it does

`GET /data/quality` reports, over the whole `research_papers` table: a completeness score, a consistency score, an estimated duplicate rate, and a per-source paper count.

How it is implemented (`app/routers/quality.py`)

```
completeness = (papers_with_abstract≥50chars + papers_with_doi + papers_with_year +
papers_with_title≥5chars) / (4 × total_papers) consistency = (# papers where 1950 ≤ year ≤ 2030
AND 50 ≤ len(abstract) ≤ 10000) / total_papers duplicate_rate = 1 − (# unique lower-cased titles /
total_papers)
```

This is a straightforward completeness/consistency/duplication audit — deliberately simple and fully transparent, since its purpose is to give the team (not the ML pipeline) visibility into corpus health before it's used to train or index anything downstream.

8 · Shared Infrastructure — SLIIT Paper Index

`app/services/paper_index.py` is the one piece of infrastructure reused by three separate features in this module (categorization's "related papers", plagiarism trend search's "related SLIIT papers", and available for the summarizer). It lazily loads `data/papers_raw_sliit.json` (4,219 records) and either reads a pre-computed `data/paper_embeddings.npy` matrix, or — only if that file is missing — encodes every paper's `"{title}. {abstract[:800]}"` on the fly (logged as taking ~90 seconds), then caches the resulting 4219×384 float32 matrix in-memory for the life of the process. Every subsequent `find_related()` call is then a single normalised matrix multiply (~30 ms), with optional year-range and subject-substring filtering applied by zeroing out non-matching rows before ranking (`np.where(mask, sims, -1.0)`) rather than re-slicing the embeddings matrix.

9 · Planned vs. Production Models

Spec model	File	Status	What runs in production instead
SciBERT multi-label classifier (encoder + dense head, BCE loss, target Macro F1 ≥0.80)	`app/models/topic_classifier.py`	Present but not imported by the router	TF-IDF + One-vs-Rest LogReg (Feature B, `app/services/topic_classifier.py`)
BERTopic (SBERT + UMAP n_neighbors=15 + HDBSCAN min_cluster_size=10 + c-TF-IDF) for topic discovery	`app/models/bertopic_model.py`	Present but not imported by any router	Pre-computed SBERT topic buckets consumed directly by plagiarism-trend search (Feature D)
Research summarizer — `facebook/bart-large-cnn` / `flan-t5-base` + LoRA (r=16, α=32), target ROUGE-1≥0.45	`app/models/summarizer.py`	Present but not imported by the router	Extractive SBERT centroid+MMR summarizer (Feature C, `app/services/extractive_summarizer.py`)

> **Why this matters for the report:** in every case the classical/retrieval-based production implementation was chosen for the same combination of reasons — no GPU inference budget on the deployment target (Railway), zero-hallucination requirements for academic content, and CPU-only latency suitable for a request/response API rather than a batch job. The deep-learning wrapper classes and their training scripts (`services/module3-data/training/`) remain in the repository as the documented, trainable path envisioned by the original spec.

10 · Endpoint Reference

Method & Path (via gateway `/api/v1/module3/…`)	Feature	Backing service
POST `/data/scrape`	Start scrape job	scrapers/*.py
GET `/data/scrape/{job_id}`	Job status	pipeline.py (in-memory jobs)
POST `/data/categorize`	Topic categorization	topic_classifier.py (+ Gemini fallback)
GET `/data/categorize/status`	Model health	topic_classifier.get_model_info
POST `/data/summarize`	Summarize text	extractive_summarizer.py (+ Gemini fallback)
POST `/data/summarize/upload`	Summarize PDF	pdf_extractor.py + extractive_summarizer.py
POST `/data/summarize/report`	HTML report download	summarizer.py (render only)
GET `/data/summarize/status`	Model health	extractive_summarizer.get_model_info
GET `/data/plagiarism-trends`	Legacy Supabase trend rows	Supabase (plagiarism_trends)
POST `/data/plagiarism-trends/search`	Topic-aware trend search	plagiarism_analyzer.py + paper_index.py
POST `/data/plagiarism-trends/search/report`	HTML report download	plagiarism_analyzer.py
POST `/data/plagiarism-trends/compare`	Compare two texts	plagiarism_analyzer.compare_papers
POST `/data/plagiarism-trends/compare-pdf`	Compare two PDFs	pdf_extractor.py + plagiarism_analyzer.py
POST `/data/plagiarism-trends/compare/report`	HTML report download	plagiarism_analyzer.py
GET `/data/plagiarism-trends/status`	Model health	plagiarism_analyzer.get_model_info
GET `/data/quality`	Corpus quality metrics	quality.py + Supabase